Agora — React Native Video Calling App

A React Native video/audio calling app that uses **Agora** for real-time media, **Firebase** for user presence and push delivery, and **AWS Lambda** as a lightweight backend to send incoming-call notifications.

Table of Contents

1. Architecture Overview
2. Prerequisites
3. Project Setup
4. Agora Account & App ID
5. Firebase Full Configuration
    - 5.1 Create Firebase Project
    - 5.2 Firestore Database
    - 5.3 Realtime Database (RTDB)
    - 5.4 Firebase Cloud Messaging (FCM)
    - 5.5 Android — google-services.json
    - 5.6 iOS — GoogleService-Info.plist
    - 5.7 Firebase Admin SDK Service Account
6. AWS Lambda Setup
7. Configure src/config.ts
8. Apple Push Notifications (APNs) for iOS
9. Android Notifications
10. Running the App
11. Lambda — Update & Redeploy
12. Troubleshooting
13. File Reference

---

1. Architecture Overview

```
┌──────────────────────────────────────────────────────────────┐
│                          React Native App                    │
│                                                              │
│  SetupScreen  registers userId + fcmToken in Firestore       │
│  HomeScreen   shows peer list (Firestore) + presence (RTDB)  │
│               calls Lambda to send FCM to callee             │
│  CallScreen   joins Agora RTC channel (video/audio)          │
│               Agora RTM used as fallback in-app signaling    │
└──────────────────────────────────────────────────────────────┘
         │                        │                  │
         ▼                        ▼                  ▼
  Firebase Firestore        Firebase RTDB       AWS Lambda
  (users collection)        (presence/uid)      (sends FCM)
         │                                          │
         └──────────────── Firebase Admin SDK ──────┘
```

```
                                                    |
                                                    ▼
                                              FCM → APNs/GCM
                                          (push to callee device)
```

Call flow step by step:

1. Both users open the app and enter a display name → saved to Firestore `users/{uid}` with their FCM token.
2. Presence is tracked in RTDB (`presence/{uid}`) using `onDisconnect` — the server automatically flips the status to `offline` when the socket drops.
3. Caller taps a user card → app sends a `POST` to the Lambda URL with `callerUserId`, `calleeUserId`, `channelName`, and `callerDisplayName`.
4. Lambda looks up the callee's `fcmToken` from Firestore and sends an FCM **data message**.
5. Callee device wakes up from the FCM push → app shows an incoming-call card with Accept / Decline.
6. Both devices join the same Agora RTC channel identified by `channelName`.
7. Agora RTM is also subscribed to a shared channel (`call-signaling`) as a secondary signaling path for when both devices are active on screen simultaneously.

---

2. Prerequisites

Tool	Minimum version	Notes
Node.js	18	Required by the project's `engines` field
npm or yarn	any recent	Used for JS dependencies
React Native CLI	0.75.x	Matches `react-native` version in package.json
Xcode	15+	Required for iOS builds
Android Studio	any recent	Required for Android builds; API 23 minimum
Ruby + Bundler	any recent	Required by CocoaPods
CocoaPods	1.14+	iOS dependency manager
AWS CLI	any	For deploying Lambda from the terminal
Firebase CLI	any	For deploying RTDB rules from the terminal

---

3. Project Setup

3.1 Clone and install

```
git clone <YOUR_REPO_URL>
cd Agora

# Install JS dependencies (postinstall will auto-apply patches)
```

```
npm install
# or
yarn install
```

> The `postinstall` script runs `patch-package` automatically. This applies the patch in `patches/agora-react-native-rtm+2.2.6.patch`, which fixes a native library conflict between the Agora RTM and RTC SDKs. See §12 for details.

3.2 Create your local config file

```
cp src/config.example.ts src/config.ts
```

`src/config.ts` is git-ignored. You will fill in its two values in later steps.

3.3 iOS — install CocoaPods

```
cd ios
bundle exec pod install   # preferred: uses the Gemfile-locked pod version
# or if you don't have bundler:
pod install
cd ..
```

3.4 Android

No extra steps. Gradle downloads all dependencies on the first build.

---

4. Agora Account & App ID

This project runs in **testing mode** — no token server is required. You only need an App ID.

Step 1 — Create an Agora account

Go to https://console.agora.io and sign up for a free account.

Step 2 — Create a project

1. Click **New Project** in the Agora Console.
2. Enter a project name (e.g. `agora-rn-test`).
3. Under **Authentication Mechanism**, select **Testing** (no token).

   > **Why Testing mode?** The "Secured" option requires you to run a token server that generates short-lived tokens for each channel join. For this POC, "Testing" mode skips all of that — the app passes an empty string `''` as the token when joining a channel, and Agora accepts it.

4. Click **Submit**.

Step 3 — Copy your App ID

After creation you are taken to the project detail page. Copy the **App ID** (a 32-character hex string such as `c30b99d2a68640148f25a0c2f958969f`).

> The same App ID is used for both:
>
> - **Agora RTC** (`react-native-agora`) — for the actual video/audio call
> - **Agora RTM** (`agora-react-native-rtm`) — for in-app signaling

You will paste it into `src/config.ts` in §7.

---

5. Firebase Full Configuration

5.1 Create Firebase Project

1. Go to https://console.firebase.google.com.
2. Click **Add project**.
3. Enter a project name (e.g. `react-native-agora`).
4. Disable Google Analytics if not needed.
5. Click **Create project** and wait for it to be provisioned.

5.2 Firestore Database

Firestore stores the user registry — display names and FCM device tokens. The Lambda function reads FCM tokens from here when initiating a call.

Enable Firestore

1. In the Firebase Console, go to **Build → Firestore Database**.
2. Click **Create database**.
3. Choose **Start in production mode** (you will set rules manually below).
4. Select a region close to your users.
5. Click **Enable**.

Set Firestore Security Rules

Open **Firestore Database → Rules** and replace the contents with:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read:  if true;
      allow write: if true;
    }
```

```
    }
  }
```

Click **Publish**.

> These rules are intentionally open for a test project. In production, tighten them so only authenticated users can write their own document.

Data structure written by the app

Each user's registration creates one document automatically:

```
users/
  {userId}/           ← random UUID generated on first launch
    displayName: "Alice"
    fcmToken:    "fXyZ..."   ← Firebase Cloud Messaging token for this
device
```

5.3 Realtime Database (RTDB)

RTDB powers the online/offline presence indicators shown on each user card.

Enable RTDB

1. Go to **Build → Realtime Database**.
2. Click **Create Database**.
3. Choose a region and select **Start in locked mode**.
4. Click **Enable**.

Set RTDB Rules

Open **Realtime Database → Rules** and replace the contents with:

```
{
  "rules": {
    "presence": {
      ".read": true,
      "$uid": {
        ".write": true
      }
    }
  }
}
```

Click **Publish**.

> `.read: true` on `presence` lets all clients see who is online. `$uid: { ".write": true }` lets each client write to its own presence node without authentication. This is correct for a test project.

How presence works in the app:

The app writes `{ "state": "online" }` to `presence/{userId}` on connect and `{ "state": "offline" }` on disconnect. An `onDisconnect` hook is registered with Firebase so the server side automatically flips the value if the socket drops unexpectedly (app killed, network lost, device off).

Alternative: deploy rules via Firebase CLI

```
npm install -g firebase-tools
firebase login

# From the project root (database.rules.json is already there)
firebase deploy --only database --project YOUR_FIREBASE_PROJECT_ID
```

The `database.rules.json` file at the repo root contains the exact rules above.

5.4 Firebase Cloud Messaging (FCM)

FCM is enabled automatically for every Firebase project. No extra steps are needed in the console for Android. For iOS you must upload an APNs key — covered fully in §8.

5.5 Android — google-services.json

1. Firebase Console → **Project settings** (gear icon) → **General** tab.
2. Scroll to **Your apps** → click **Add app** → choose the **Android** icon.
3. Enter the Android package name: `com.agora`
4. Optionally add a nickname. Skip the SHA-1 for now (not needed for FCM).
5. Click **Register app**.
6. Click **Download google-services.json**.
7. Place the file at: `android/app/google-services.json`

> This file is git-ignored. Every developer must download and place it themselves.

`@react-native-firebase` automatically applies the `google-services` Gradle plugin through its autolinking mechanism — you do **not** need to manually add `classpath 'com.google.gms:google-services'` or `apply plugin: 'com.google.gms.google-services'` to your build files.

5.6 iOS — GoogleService-Info.plist

1. Firebase Console → **Project settings → General → Your apps**.
2. Click **Add app** → choose the **iOS** icon.
3. Enter the iOS Bundle ID.

   > To find your bundle ID: open `ios/Agora.xcworkspace` in Xcode → select the `Agora` target → **Signing & Capabilities** tab → **Bundle Identifier**. The value used in this project is

> `com.alimotech.agora`.

4. Click **Register app**.
5. Click **Download GoogleService-Info.plist**.
6. Place the file at: `ios/Agora/GoogleService-Info.plist`

> This file is git-ignored. Every developer must download and place it themselves.

Add it inside Xcode (required):

1. Open `ios/Agora.xcworkspace` in Xcode.
2. In the Project Navigator, right-click the `Agora` folder (the blue folder icon).
3. Select **Add Files to "Agora"**.
4. Select `GoogleService-Info.plist`.
5. Check **Copy items if needed**.
6. Make sure the `Agora` target is checked.
7. Click **Add**.

5.7 Firebase Admin SDK Service Account

The Lambda function uses the Firebase Admin SDK to read Firestore and send FCM messages. This requires a **service account key** — a JSON credential file that grants server-side access to your Firebase project.

What this file is

The file `react-native-agora-firebase-adminsdk-fbsvc-de2575b240.json` in the project root is an example of a service account key. It is git-ignored because it contains a private key. **Never commit this file.**

The filename pattern is:

```
{project-id}-firebase-adminsdk-{suffix}-{hash}.json
```

How to generate your own

1. Firebase Console → **Project settings** (gear icon) → **Service accounts** tab.
2. Make sure **Firebase Admin SDK** is selected in the left pane.
3. Click **Generate new private key**.
4. Click **Generate key** in the confirmation dialog.
5. A JSON file is downloaded to your computer. Save it somewhere safe.

The file looks like this:

```
{
  "type": "service_account",
```

```
  "project_id": "your-project-id",
  "private_key_id": "abc123...",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY---
--\n",
  "client_email": "firebase-adminsdk-xxxxx@your-project-
id.iam.gserviceaccount.com",
  "client_id": "12345...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "https://...",
  "universe_domain": "googleapis.com"
}
```

You will use the **entire contents** of this file as the `FIREBASE_SERVICE_ACCOUNT` environment variable in your Lambda function (see §6, Step 3).

---

6. AWS Lambda Setup

The Lambda function at `lambda/index.js` is a thin backend that:

1. Receives a POST request from the app.
2. Looks up the callee's `fcmToken` from Firestore.
3. Sends an FCM data-push to the callee's device.

Step 1 — Create a Lambda function

1. Log in to the AWS Console.
2. Navigate to **Lambda → Functions → Create function**.
3. Choose **Author from scratch**.
4. Configure:
   - **Function name**: `agora-func` (or any name)
   - **Runtime**: `Node.js 22.x` (or 20.x)
   - **Architecture**: `x86_64`
5. Click **Create function**.

Step 2 — Enable a Function URL

A Function URL gives the Lambda a stable HTTPS endpoint without needing API Gateway.

1. Inside your Lambda function, go to the **Configuration** tab.
2. Click **Function URL** in the left sidebar.
3. Click **Create function URL**.
4. Set:
   - **Auth type**: `NONE` (no IAM auth — appropriate for a test project)
   - **CORS**: Enable it; add `*` to **Allowed origins**
5. Click **Save**.
6. Copy the **Function URL** — it looks like:

```
https://xxxxxxxxxxxxxxxxxxxxxxxx.lambda-url.eu-north-1.on.aws/
```

You will paste this into `src/config.ts` as `LAMBDA_URL`.

Step 3 — Set the environment variable

The Lambda reads your Firebase service account from an environment variable so that credentials are never hardcoded in the source code.

1. **Configuration tab → Environment variables → Edit → Add environment variable**.
2. Key: `FIREBASE_SERVICE_ACCOUNT`
3. Value: paste the **entire JSON content** of your service account file as a single string.

   Run this command locally to get the correctly collapsed single-line value:

   ```
   cat your-adminsdk-file.json | tr -d '\n'
   ```

   Copy the output and paste it as the value.
4. Click **Save**.

Step 4 — Package and upload the Lambda

```
# From the project root
cd lambda

# Install production dependencies
npm install --omit=dev

# Create the deployment zip
zip -r ../agora-func.zip index.js node_modules/

cd ..
```

Upload via the AWS Console:

1. In your Lambda function, click the **Code** tab.
2. Click **Upload from → .zip file**.
3. Select `agora-func.zip`.
4. Click **Save**.

Or upload via AWS CLI:

```
aws lambda update-function-code \
  --function-name agora-func \
  --zip-file fileb://agora-func.zip \
  --region YOUR_REGION
```

Step 5 — Verify the handler setting

1. Click **Code → Runtime settings → Edit**.
2. Confirm **Handler** is set to `index.handler`.
3. Click **Save** if you changed it.

Step 6 — Test the Lambda

In the Lambda console, go to the **Test** tab. Create a new test event with this body:

```
{
  "requestContext": {
    "http": {
      "method": "POST"
    }
  },
  "body": "{\"callerUserId\":\"user-a\",\"calleeUserId\":\"user-
b\",\"channelName\":\"test-channel\",\"callerDisplayName\":\"Alice\"}"
}
```

Click **Test**. Expected responses:

Response	Meaning
`{"ok": true}`	Everything works — FCM notification was sent
`{"error": "Callee not found"}`	`user-b` does not exist in Firestore yet (expected in a fresh test)
`{"error": "Invalid JSON"}`	The body format is wrong
`{"error": "Internal server error"}`	Check CloudWatch logs (see §12)

---

7. Configure src/config.ts

After completing §4 (Agora) and §6 (Lambda), fill in both values:

```
// src/config.ts  — do NOT commit this file
export const AGORA_APP_ID = 'your_32_character_agora_app_id_here';
```

```
export const LAMBDA_URL   = 'https://your-function-url.lambda-
url.REGION.on.aws/';
```

`src/config.ts` is git-ignored. Use `src/config.example.ts` as the template. The example file already has the correct shape:

```
// src/config.example.ts
export const AGORA_APP_ID = 'YOUR_AGORA_APP_ID';
export const LAMBDA_URL = 'https://YOUR_LAMBDA_FUNCTION_URL/';
```

---

8. Apple Push Notifications (APNs) for iOS

FCM on iOS is delivered through Apple's APNs. Without this configuration, iOS devices will **not** receive incoming-call notifications.

8.1 What is the .p8 key file?

`AuthKey_UZX8LP8Q72.p8` is an **APNs Authentication Key** — a private key issued by Apple that lets Firebase communicate with Apple's push notification servers on your behalf.

The filename encodes the **Key ID**: `AuthKey_{KEY_ID}.p8`
In this project the Key ID is `UZX8LP8Q72`.

This file is git-ignored and must be generated per Apple Developer account.

8.2 Generate your own APNs Auth Key

1. Log in to https://developer.apple.com/account.
2. Go to **Certificates, Identifiers & Profiles → Keys** (left sidebar).
3. Click the **+** button.
4. Enter a key name (e.g. `FCM Push Key`).
5. Check **Apple Push Notifications service (APNs)**.
6. Click **Continue → Register**.
7. Click **Download**. The file is named `AuthKey_{YOUR_KEY_ID}.p8`.

   > **Critical:** You can only download this key **once**. If you lose it, you must revoke it and create a new one.

8. Note the **Key ID** shown on the confirmation page (the same 10-character string that appears in the filename).
9. Note your **Team ID** — find it at developer.apple.com/account under the **Membership** section.

8.3 Upload the APNs Key to Firebase

1. Firebase Console → **Project settings → Cloud Messaging** tab.
2. Scroll to the **Apple app configuration** section.
3. Under **APNs Authentication Key**, click **Upload**.
4. Select your `.p8` file.
5. Enter your **Key ID** (e.g. `UZX8LP8Q72`).
6. Enter your **Team ID** (10-character Apple Developer Team ID).
7. Click **Upload**.

After this, Firebase can deliver push notifications to iOS devices using APNs.

8.4 Configure capabilities in Xcode

1. Open `ios/Agora.xcworkspace` in Xcode.

   > Always open `.xcworkspace`, **never** `.xcodeproj`, after running `pod install`.

2. Select the **Agora** project in the Project Navigator (left panel).

3. Select the **Agora** target (not the project, the target) under **TARGETS**.

4. Go to the **Signing & Capabilities** tab.

5. Click **+ Capability** (top-left of the tab).

   - Add **Push Notifications**.

6. Click **+ Capability** again.

   - Add **Background Modes**.

7. In the **Background Modes** section that now appears, check both:

   - **Audio, AirPlay, and Picture in Picture**
   - **Remote notifications**

   > **Why Remote notifications?** The Lambda sends a **silent push** (`content-available: 1`) to wake the app in the background. Without this background mode enabled, iOS will not wake the app and the incoming-call UI will never appear.

8.5 Add [FIRApp configure] to AppDelegate.mm

`@react-native-firebase` v21 requires Firebase to be initialised in the native app delegate. Open `ios/Agora/AppDelegate.mm` and modify it as follows:

```
#import "AppDelegate.h"
#import <React/RCTBundleURLProvider.h>
#import <Firebase.h>          // ← ADD THIS LINE

@implementation AppDelegate
```

```
- (BOOL)application:(UIApplication *)application
    didFinishLaunchingWithOptions:(NSDictionary *)launchOptions
{
  [FIRApp configure];          // ← ADD THIS LINE (before [super ...])
  self.moduleName = @"Agora";
  self.initialProps = @{};
  return [super application:application
didFinishLaunchingWithOptions:launchOptions];
}

// ... rest of the file unchanged
```

Without `[FIRApp configure]`, Firebase will not initialise on iOS and you will see a crash or FCM will silently fail to deliver notifications.

8.6 Request notification permission from JavaScript

In your app code, request permission before trying to get an FCM token:

```
import messaging from '@react-native-firebase/messaging';

async function requestPushPermission() {
  const authStatus = await messaging().requestPermission();
  const enabled =
    authStatus === messaging.AuthorizationStatus.AUTHORIZED ||
    authStatus === messaging.AuthorizationStatus.PROVISIONAL;
  if (enabled) {
    const token = await messaging().getToken();
    return token;
  }
  return null;
}
```

> On a **simulator**, FCM tokens are **not available**. You must test push notifications on a **real physical iOS device**.

---

9. Android Notifications

On Android, FCM works natively without APNs. The Lambda sends a high-priority message (`android: { priority: 'high' }`), which wakes Android devices even in Doze mode.

No extra configuration is needed beyond placing `google-services.json` in `android/app/` as described in §5.5.

All required permissions are already declared in `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
```

---

10. Running the App

Complete all configuration steps before running for the first time.

Start Metro bundler

```
npx react-native start
# or
yarn start
```

Run on iOS

```
npx react-native run-ios

# Target a specific simulator:
npx react-native run-ios --simulator "iPhone 15 Pro"
```

Run on Android

Connect a device or start an emulator, then:

```
npx react-native run-android
```

What happens on first launch

1. A unique `userId` (UUID) is generated and saved on the device.
2. The app requests FCM permission from the OS (iOS shows a system dialog).
3. The **Setup screen** appears — enter a display name and tap **Continue**.
4. The app writes `{ displayName, fcmToken }` to Firestore `users/{userId}`.
5. The **Home screen** loads and shows other registered users.
6. Online/offline status (green/grey dot) is driven by RTDB presence in real time.
7. Tapping a user card initiates a call — the Lambda is called, FCM push is sent, and both devices join the Agora RTC channel.

---

11. Lambda — Update & Redeploy

When you modify `lambda/index.js` or update its dependencies:

```
cd lambda

# Reinstall if package.json changed
npm install --omit=dev

# Repackage
zip -r ../agora-func.zip index.js node_modules/

cd ..

# Deploy via AWS CLI
aws lambda update-function-code \
  --function-name agora-func \
  --zip-file fileb://agora-func.zip \
  --region YOUR_REGION

# Confirm the update timestamp
aws lambda get-function-configuration \
  --function-name agora-func \
  --region YOUR_REGION \
  --query 'LastModified'
```

Update the FIREBASE_SERVICE_ACCOUNT environment variable

If you rotate your service account key:

```
aws lambda update-function-configuration \
  --function-name agora-func \
  --region YOUR_REGION \
  --environment "Variables={FIREBASE_SERVICE_ACCOUNT=$(cat your-adminsdk-
file.json | tr -d '\n')}"
```

View Lambda logs

```
aws logs tail /aws/lambda/agora-func --follow --region YOUR_REGION
```

Or in the AWS Console: **Lambda → Monitor → View CloudWatch logs**.

---

12. Troubleshooting

libaosl.so conflict on Android

Symptom: `UnsatisfiedLinkError` referencing `libaosl.so` at runtime, or a duplicate symbol build error.

Cause: Both `react-native-agora` (RTC) and `agora-react-native-rtm` (RTM) bundle `libaosl.so`. Loading both copies causes a conflict.

Fix already applied in this repo:

- `android/app/build.gradle` excludes the Maven RTM artifact and uses `pickFirsts` to resolve the duplicate `.so`.
- A stripped local AAR (`android/app/libs/agora-rtm-2.2.6-noaosl.aar`) replaces the Maven version — `libaosl.so` has been removed from it.
- `patches/agora-react-native-rtm+2.2.6.patch` patches the RTM build script to use this local AAR instead of the Maven artifact.
- The patch is applied automatically on `npm install` via the `postinstall` script.

If the patch is not applied, run:

```
npx patch-package
```

iOS — aosl.xcframework embed conflict

Symptom: CocoaPods build error about a duplicate `aosl.xcframework` or embedding conflict between pods.

Fix already applied: The same patch changes the RTM podspec to depend only on `AgoraRtm/RtmKit` (excluding the `aosl` framework), since `react-native-agora` already provides it via `AgoraVideo_Special_iOS`.

FCM token not received on iOS

- Make sure §8 is completed fully: APNs key uploaded to Firebase, Push Notifications capability added in Xcode, Remote notifications background mode enabled, `[FIRApp configure]` added to `AppDelegate.mm`.
- FCM tokens are **not issued on iOS Simulators**. Test on a real physical device.
- Always open `Agora.xcworkspace` (not `Agora.xcodeproj`) in Xcode after running `pod install`.

Firestore permission denied on setup

Error shown in app: "Firestore blocked saving your profile (permission denied)"

Check that you published the Firestore rules from §5.2. In the Firebase Console go to **Firestore Database → Rules** and confirm the `users/{userId}` path allows reads and writes.

Lambda returns 404 "Callee not found"

The callee's document does not exist in Firestore. This is expected when:

- The callee has not opened the app and completed the setup screen yet.

- The `userId` passed to the Lambda does not match the Firestore document ID.

Lambda returns 500 "Internal server error"

Check CloudWatch logs for the exact error. Common causes:

- `FIREBASE_SERVICE_ACCOUNT` is missing or is malformed JSON (not a valid string).
- The service account was revoked in the Firebase Console.
- The `project_id` in the service account does not match the Firebase project containing the `users` collection.

RTM spinner never resolves ("Signaling connecting")

- The `AGORA_APP_ID` in `src/config.ts` is wrong or empty.
- The device has no internet connection.
- Agora RTM and Agora RTC must use the same App ID. Double-check you copied the right project's ID from the Agora Console.

iOS build fails after pod install

Make sure you opened `Agora.xcworkspace` — not `Agora.xcodeproj`.

If pods are out of sync after changing the `package.json`:

```
cd ios
pod deintegrate
pod install
cd ..
```

Android build fails with "Duplicate class"

```
cd android
./gradlew app:dependencies | grep "agora-rtm"
```

If you see the Maven `io.agora:agora-rtm` appearing (instead of the local AAR), the patch did not apply correctly. From the project root, run:

```
npx patch-package
```

---

13. File Reference

File	Purpose	Committed?
`src/config.ts`	Agora App ID + Lambda URL	**No** — copy from `config.example.ts`
`src/config.example.ts`	Template for `config.ts`	Yes
`android/app/google-services.json`	Firebase Android config	**No** — download from Firebase Console
`ios/Agora/GoogleService-Info.plist`	Firebase iOS config	**No** — download from Firebase Console
`*-firebase-adminsdk-*.json`	Firebase Admin SDK service account key	**No** — generate from Firebase Console
`AuthKey_XXXXXXXXXX.p8`	Apple APNs authentication key	**No** — generate from Apple Developer portal
`lambda/index.js`	AWS Lambda function source code	Yes
`lambda/package.json`	Lambda dependency manifest	Yes
`database.rules.json`	Firebase RTDB security rules	Yes
`patches/agora-react-native-rtm+2.2.6.patch`	Native SDK conflict fix (RTM + RTC)	Yes
`android/app/libs/agora-rtm-2.2.6-noaosl.aar`	Stripped Agora RTM AAR (no `libaosl.so`)	Yes